University of Bamberg

Chair of Explainable Machine Learning

Deep Learning Exercise

Assignment 1



Term:
WS 26/27

Instructor:
Prof. Dr. Christian Ledig

Assignment Start:
Monday, October 12, 2026 at 8:00AM

Assignment End:
Sunday, November 8, 2026 at 11:59PM



Contents

Guidelines 4

Related Classes and Resources 5

Necessary Tools and Those That Make Your Life Easier 6

Initializing Your Tools 8

Submission Instructions 11

Theory [40 points + 10 points bonus] 13

1 Derivatives [8 points] 14
1.1 Partial Derivative [2 points] . . . 14
1.2 Derivative of the Sigmoid Function [4 points] . . . 15
1.3 Weight Matrix [2 points] . . . 16

2 Linear Regression [2 points] 17

3 Optimization [15 points + 10 points bonus] 18
3.1 Level Surface Set [10 points] . . . 18
3.2 Bonus: Relation Gradient - Local Minimum [5 points] . . . 19
3.3 Bonus: Relation Gradient - Global Minimum [5 points] . . . 20
3.4 Gradient of the Softmax Function [5 points] . . . 21

4 Directed Acyclic Graph [10 points] 22
4.1 G is a DAG then G has a Topological Ordering [2 points] . . . 22
4.2 G has a Topological Ordering then G is a DAG [2 points] . . . 23
4.3 Computational Graphs and Backpropagation [6 points] . . . 24

5 Logistic Regression [5 points] 26

Code [60 points + 5 points bonus] 27

Overview 27

Testing 27

6 Numpy Basics [12 points] 29
6.1 Basic Numpy Functions [12 points] . . . 29

7 Numpy Applied [11 points + 5 points bonus] 38
7.1 Simple Array Border Pattern [1 point] . . . 38
7.2 Checkerboard Pattern [3 points] . . . 38
7.3 Shapes [7 points] . . . 42
7.4 Bonus: Color Spectrum [5 points] . . . 46

8 Torch Tensors Basics [11.5 points] **48**
8.1 Install Torch [0 point] . . . 48
8.2 Tensor Basics [4 points] . . . 48
8.3 Tensor Calculations [2.5 points] . . . 53
8.4 Tensor Statistics [1.5 points] . . . 57
8.5 Tensor Operations [3.5 points] . . . 60

9 Derivatives and Graphs [5.5 points] **62**
9.1 Plotting [0 point] . . . 62
9.2 Derivatives [3.5 points] . . . 62
9.3 Partial Derivatives [2 points] . . . 64

10 Linear Regression [7 points] **65**
10.1 Visualization [0 point] . . . 66
10.2 Data Loading [1 point] . . . 66
10.3 Iterative Linear Regression [3 points] . . . 67
10.4 Testing on a Real World Example [3 points] . . . 71

11 Logistic Regression [13 points] **75**
11.1 Data Loading [3 points] . . . 76
11.2 Preprocessing [2 points] . . . 78
11.3 Building the Algorithm [7 points] . . . 79
11.4 Viewing and Evaluating the Results [1 point] . . . 83

Guidelines

The chapters up to the start of the theory part (Guidelines, Resources, Tools and Submission Instructions) apply to **all four assignments**. Assignments 2 to 4 do not repeat them; they open with a one-page summary and refer back here.

- We generally encourage you to collaborate with other students. You may talk to a friend or create a post in VC to discuss the questions and potential directions for solving them. However, this is not a group assignment. Hence, each student must write his/her/their own answers and explicitly mention any collaborators.
- We will apply anti-cheating software to check the submissions for plagiarism. Besides all submissions of the current semester, we further check for online resources as well as submissions of previous semesters. Anyone who is flagged by the software will automatically receive 0 points for the homework. If you feel that you were accused wrongly, please approach your TA immediately.
- Do NOT change the filenames and function definitions in the skeleton code provided. This will cause the test scripts to fail and results in you receiving 0 points for the respective task. You may also NOT change the import modules in each file or import additional modules.
- It is your responsibility to make sure that all code and other deliverables are in the correct format and that your submission compiles and runs. Failing to do so will result in the reduction of points.
- It is your responsibility to hand in the submission in time. Anyone failing to do so will automatically receive 0 points for the homework. If you were not able to submit your work due to illness or other reasons, please approach your TA immediately to clarify further action.
- We hope you enjoy your dive into the area of Deep Learning with its broad range of applications. Deep Learning is considered the driver of AI with its limitations having yet to be explored. Please have fun coding and solving the challenging questions, so you don't remain the dinosaur, Mark Cuban labeled you as back in 2017.

Related Classes and Resources

If an explanation in this course does not land, a second one often does. These are the sources this exercise draws on, and the ones students in earlier years came back to.

Where our assignments come from. Parts of the programming tasks are adapted from two courses:

- Deep Learning (CS 7643), Georgia Institute of Technology
- Deep Learning, Eberhard Karls Universität Tübingen

Courses with material online.

- CS231n: Deep Learning for Computer Vision, Stanford. The lecture notes are the single best written introduction to convolutional networks and backpropagation.
- Introduction to Deep Learning (11-785), CMU
- Deep Learning, New York University
- Deep Learning, FAU Erlangen-Nuremberg
- Deep Learning Specialization, DeepLearning.AI, and Deep Neural Networks with PyTorch, IBM. Both are paid courses that can be audited for free.

Books and lecture series.

- Deep Learning by Goodfellow, Bengio and Courville. The standard reference, free to read online. Chapters 6 to 9 cover most of this course.
- Dive into Deep Learning. A textbook whose every example is runnable code, in PyTorch among others. Useful if you learn by changing something and seeing what happens.
- Neural Networks: Zero to Hero by Andrej Karpathy. Builds backpropagation and then a small language model from scratch, on video, one line at a time. Closest in spirit to our assignments 1 and 4.
- Hugo Larochelle's Neural Networks class. Older (2013) and still one of the clearest treatments of the fundamentals.
- Machine Learning, Oxford by Nando de Freitas. Also older (2014/15), and strong on the probabilistic view that our theory questions assume.

Necessary Tools and Those That Make Your Life Easier

You need three things: a **Python 3** distribution, something to edit and run **Jupyter notebooks** in, and access to a **GPU** for the larger tasks. The sections below give one recommended option for each, plus the alternatives students in previous years have used. Any combination works. Nothing in the assignments depends on a particular editor.

Visual Studio Code

Visual Studio Code is free, runs on Windows, macOS and Linux, and opens Jupyter notebooks directly, which is the format of every programming task in this course. Install it, then add the two extensions `Python` and `Jupyter` from the Extensions panel (`Ctrl+Shift+X`, or `Cmd+Shift+X` on macOS). That is the whole setup.
It gives you what you actually need while working on the assignments: jump to a definition, find every use of a name across the project, a debugger so you do not have to reason in print statements, and an integrated terminal for the test runner.

PyCharm

PyCharm (by JetBrains) is the heavier alternative, and a good one if you like a full IDE. The free community edition covers everything this course needs. JetBrains also offers free educational licenses, so as a student you can use the professional edition at no cost for the duration of your studies.

Jupyter Notebooks

Every programming task in this course is a Jupyter notebook. A notebook interleaves code, formatted text, mathematics and plots in one document, which is why the tasks are written in that form: the explanation, the code you write and the output you produce sit next to each other. `VS Code` and the professional edition of `PyCharm` both open notebooks directly. You can also run the notebook server in your browser, which needs nothing beyond the environment you create below:

```
$ conda activate xai-dl-a1
$ jupyter notebook
```

Google Colab

Google Colab runs notebooks in the cloud on a free GPU. It is the answer if you have no GPU of your own, and the assignments are written so that this is a real option rather than a fallback. All you need is a free Google account, since `Google Colab` stores its notebooks in `Google Drive`.
The free tier limits how long a session may run and how much memory it gets. Nothing in this course comes close to those limits, but note that a session left idle

is recycled, and anything not saved to your Drive is lost with it. If you have never used Colab, work through the official introduction notebook first. The commands you will need are listed at the end of this chapter.

Initializing Your Tools

Python and Dependencies

We work with **Python 3**. If you have never used Python, study one of the free tutorials first, for example the Official Python Tutorial or the W3 Schools Python Tutorial.
If you do not have a Python distribution yet, install Miniforge. It is a minimal installer for `conda` that uses the community-maintained `conda-forge` channel, which is the channel our environment files declare. The conda user guide explains environments in more detail.
Each assignment ships a `environment.yaml` listing the libraries that assignment needs, at the versions the tests were written against. Open a terminal in the assignment's `Programming` folder and run:

```
$ conda env create -f environment.yaml
$ conda activate xai-dl-a1
```

Activate that environment every time you work on the assignment, and select it as the kernel when you open a notebook. Most "this import fails" and "my numbers differ from the expected ones" problems are a notebook running against the wrong interpreter.
If you prefer to manage your own environment, read `environment.yaml` for the list of libraries and their versions.

PyTorch

`environment.yaml` installs the **CPU** build of `PyTorch`. Implementing and testing every task in this course works on a CPU, so that is the default and you need to do nothing. Only the training sections of assignment 4 are much faster on a GPU; if you have none, those sections point you to the free GPU on `Google Colab`.
If you have an NVIDIA GPU and want to use it, replace the one line

```
- pytorch-cpu=2.13
```

in `environment.yaml` with

```
- pytorch-gpu=2.13
```

before creating the environment. On other hardware, or if you would rather install with pip, take the exact command from the official selector on pytorch.org. Keep the version, so that your results match what the tests expect.
Assignment 2 needs no `PyTorch` at all and does not install it.

Setting up VS Code

1. **File** → **Open Folder** and select your local **Assignment 1** copy.

2. Open the command palette (`Ctrl+Shift+P`, or `Cmd+Shift+P` on macOS), run **Python: Select Interpreter** and pick the **xai-dl-a1** environment. VS Code remembers this per folder.

3. Open one of the notebooks. Click the kernel name in the top right and select the same **xai-dl-a1** environment. The interpreter and the notebook kernel are two separate settings, and getting only one of them right is the most common cause of a failing import.

Setting up PyCharm

1. On the welcome screen click **New Project**. If PyCharm opens your last project instead, use **File** → **New Project**.

2. Under **Pure Python** choose **Previously configured interpreter**. If your conda environment is not in the dropdown, select **...** → **Conda Environment** and point it at the **xai-dl-a1** environment, then click **Ok**.

3. Set **Location:** to your local **Assignment 1** copy and click **Create**.

4. PyCharm asks **Directory is Not Empty**. Click **Create from Existing Sources**.

Working in Google Colab

Upload the assignment folder to your Google Drive, open the notebook from there, and set **Runtime → Change runtime type** to a GPU. The commands you need inside the notebook are:

1. Mount your Drive:

   ```
   from google.colab import drive
   drive.mount('/content/drive')
   ```

2. Change into your assignment folder, and add it to the module search path so that the assignment's own Python files can be imported:

   ```
   %cd "/content/drive/MyDrive/<your path>/Programming"
   import sys; sys.path.insert(0, ".")
   ```

   Replace the path with wherever you put the folder. Note that the mounted path is MyDrive without a space, whatever the Drive web interface shows.

3. Check that a GPU is actually attached:

   ```
   !nvidia-smi
   ```

4. Run one of the assignment's Python files with arguments:

   ```
   !python train.py --epochs 10
   ```

Submission Instructions

Besides checking your submissions by hand, we will be using automated tests, too. Please read the following instructions for submitting your solutions carefully, since failure to follow these instructions may result in parts of your assignment not being graded. We will not entertain regrading requests for failing to follow the instructions.

All exercises or questions marked with **[Bonus]** do not have to be done to achieve the maximum amount of points for their respective assignment. However, these tasks can be used to replace another task of the same assignment or guarantee that you achieve the maximum amount of points possible. All bonus tasks can only be used for their respective assignment and can not be used to improve the results in a later assignment. This means one can at max get 100 Points for each assignment.

Theory

For the theory part of the homework you have to upload one single PDF containing the answers to all the questions. Please note that the solution to each problem/subproblem must be on a separate page. This will help us to grade your submissions in less time. Please use the provided template to create your submission.

Please also state any colaborators you have exchanged your thoughts and ideas with under the section **Colaborators**. If you have worked completely on your own just enter **None** in that section.

After finishing the theory part, generate the resulting PDF and submit it via VC.

However, if you are tasked with running experiments in the programming section of this assignment, you will need to create the resulting report, add it at the end of your theory submission PDF to create a single PDF and submit this in the end via VC.
For the report you will use the provided template to collect and summarize your experimental results and findings. Simply go through all of the provided questions within the template to do so. For whichever questions asking for plots, please include plots from all your experiments.
Explanations should go into why things work the way they do with proper deep learning theory. For example, with hyperparameter tuning you should explain the reasoning behind your choices and what behavior you expected. If you need more than one slide for a question, you are free to create new slides **right after the given one**.
In the end you will export your report in PDF format and combine it with your submission of the theory questions to a single PDF before submitting the resulting PDF via VC.

Coding

For your convenience and to assure that your code can be processed by our test cases you are only allowed to write code between the `# YOUR CODE HERE` comments:

```
###########################################################
#                   START OF YOUR CODE                    #
# TODO:                                                   #
#     ...                                                 #
###########################################################
                              .
                              .
                              .
###########################################################
#                    END OF YOUR CODE                     #
###########################################################
```

In a Jupyter notebook, whether you opened it in VS Code, PyCharm or your browser, you run a cell after writing your code by either pressing `SHIFT+ENTER` or by clicking on `"Run Cell"` (denoted by a play symbol) in the upper bar of the notebook.
`It is important to note that after you modified the code within a cell you always have to run that cell again before the changes are recognized`

After passing all provided test, make sure you don't forget to upload your submission to VC. For that you will need to submit a zip file containing all your codes. For your convenience, we provide a handy script for you that does that directly. Use the "collect_submission"-script, which checks all files for completeness and zips the files you need for the upload.
Try running:

```
$ bash collect_submission.sh
```

or if you are using a Windows machine, try running:

```
$ C:\...\assignmentfolder> collect_submission.bat
```

Before submitting your assignment, please make sure you are not doing the following:

1. You have not added any *extra* `print` statement(s) in the assignment.
2. You have not added any *extra* `code cell(s)` in the assignment.
3. You have not changed any of the `function parameters`.
4. You are not using any `global variables` inside your graded exercises. Unless specifically instructed to do so, please refrain from it and use the local variables instead.
5. You are not changing the `assignment code` where it is not required, like creating extra variables.

If you do any of the mentioned, our test scripts will fail and as a result you will receive **0 points** for the respective task.

Theory [40 points + 10 points bonus]

Please use the provided template **writeup.tex** to collect your solutions to the theory questions below. Please note that the solution to each problem/subproblem must be on a separate page. Furthermore, please list any collaborators you have exchanged your thoughts and ideas with. After finishing this part, generate the resulting PDF and submit it via VC.

1 Derivatives [8 points]

1.1 Partial Derivative [2 points]

Given:

$$f(x) = \begin{pmatrix} e^{x_1} \sin(x_3) \\ \sqrt{x_2 + x_3} \end{pmatrix} \qquad \text{with } f : \mathbb{R}^3 \mapsto \mathbb{R}^2 \tag{1}$$

Calculate the matrix of partial derivatives $\frac{\partial f}{\partial x}(x)$.

1.2 Derivative of the Sigmoid Function [4 points]

a) The sigmoid function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \qquad \text{with } \sigma : \mathbb{R}^1 \mapsto \mathbb{R}^1 \tag{2}$$

Calculate $\sigma'(x)$. The solution should be simplified as much as possible and include the sigmoid function somehow...

b) With $a : \mathbb{R}^1 \mapsto \mathbb{R}^1$, we can define a function $f(x)$ as:

$$f(x) = \begin{pmatrix} a(x_1) \\ \vdots \\ a(x_n) \end{pmatrix} \tag{3}$$

Calculate the matrix of partial derivatives $\frac{\partial f}{\partial x}(x)$.

c) Replace a in the function above with the sigmoid function σ, such as:

$$f(x) = \begin{pmatrix} \sigma(x_1) \\ \vdots \\ \sigma(x_n) \end{pmatrix} \tag{4}$$

Calculate the matrix of partial derivatives $\frac{\partial f}{\partial x}(x)$.

d) What property has the resulting matrix $\frac{\partial f}{\partial x}(x)$ of task c?

1.3 Weight Matrix [2 points]

Let $\mathbf{W} \in \mathbb{R}^{m \times n}, \mathbf{x} \in \mathbb{R}^n$ and $\mathbf{y} = \mathbf{W}\mathbf{x}$.

a) Calculate the partial derivative $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$.

b) Calculate $\frac{\partial y_k}{\partial w_{ij}}$ for $k \in \{1, ..., m\}$, $i \in \{1, ..., m\}$ and $j \in \{1, ..., n\}$.

c) Let $\mathbf{A} \in \mathbb{R}^{1 \times m}$. Simplify

$$\widehat{w}_{ij} = \mathbf{A} \frac{\partial \mathbf{y}}{\partial w_{ij}} \tag{5}$$

2 Linear Regression [2 points]

The loss function $\mathcal{L}$ of a Linear Regression model with a scalar input x and scalar output y is given by:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (w_0 + w_1 x_i - y_i)^2 \tag{6}$$

In this simple 1D case, we are fitting a straight line to the dataset. This line is determined by its offset along the y-axis (w_0) and its slope (w_1). Compute the partial derivative of the loss w.r.t. the y-offset $\left(\frac{\partial \mathcal{L}}{\partial w_0}\right)$ and the partial derivative of the loss w.r.t. the slope $\left(\frac{\partial \mathcal{L}}{\partial w_1}\right)$.

3 Optimization [15 points + 10 points bonus]

3.1 Level Surface Set [10 points]

Let $f(\mathbf{x})$ be a function, where $\mathbf{x} \in \mathbb{R}^d$. The *level surface* of a function is the set of points with the same value of function, i.e.

$$L_c = \{\mathbf{x} \mid f(\mathbf{x}) = c\} \tag{7}$$

for some $c \in \mathbb{R}$.

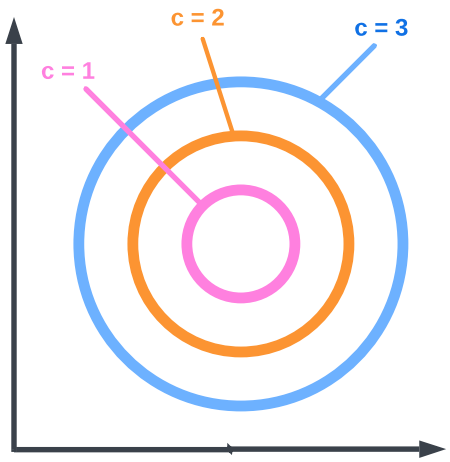


Figure 1: 2D example for level surfaces for three different values.

Figure 1 shows level surfaces for different values of c for a 2-dimensional case. The points on the perimeter of each curve illustrate the set of points sharing the same value, i.e. $f(x_1, x_2) = c$.

Recall that a gradient vector is defined as $\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, ..., \frac{\partial f}{\partial x_d}\right]$.

Consider a specific point $\mathbf{x}_0 \in \mathbb{R}^d$. Let us denote the level surface passing through this point by $L_{f(\mathbf{x_0})}$. Let us denote the gradient vector at $\mathbf{x_0}$ by ∇f_0.

Now, consider an arbitrary curve $\mathbf{r}(t) = [x_1(t); x_2(t); \ldots, x_d(t)]$ (parameterized by a scalar t) that passes through $\mathbf{x}_0$, i.e. $\mathbf{r}(t_0) = \mathbf{x_0}$ for some $t_0 \in \mathbb{R}$. Furthermore, the curve $\mathbf{r}(t)$ lies within the level surface passing through $\mathbf{x}_0$, i.e. $\mathbf{r}(t) \in L_{f(\mathbf{x_0})}$, $\forall t$.

Recall that the tangent to a curve is defined as $\frac{\partial \mathbf{r}}{\partial t}$.

With all that context and notation behind us, now comes the 'fun' part. Show that ∇f_0 is orthogonal to the tangent of $\mathbf{r}(t)$ at t_0.

Now, please describe in non-technical language what you have just proven. Why might we care about this in the context of deep learning?

3.2 Bonus: Relation Gradient - Local Minimum [5 points]

Consider a (not necessarily convex) differentiable function $g : \mathbb{R}^n \to \mathbb{R}$. g has a local minimum at some $\mathbf{w}^t$ if there exists some $\gamma > 0$ such that for all $\mathbf{w} \in \mathbb{R}^n$, $\|\mathbf{w}^t - \mathbf{w}\|_2 < \gamma \Rightarrow g(\mathbf{w}^t) \leq g(\mathbf{w})$.
Prove that if g has a local minimum at some $\mathbf{w}^t$ then the gradient at $\mathbf{w}^t = 0$, and that the converse is not necessarily true.

3.3 Bonus: Relation Gradient - Global Minimum [5 points]

Prove that if a differentiable function $g : \mathbb{R}^n \to \mathbb{R}$ is convex and the gradient at some $\mathbf{w}^*$ is 0, then $\mathbf{w}^*$ is the global minimum of g.

3.4 Gradient of the Softmax Function [5 points]

We will now consider the softmax function $\mathbf{s}(\mathbf{z})$, which takes a vector input $\mathbf{z}$ and outputs a vector whose i-th entry s_i is

$$s_i = \frac{e^{z_i}}{\sum_k e^{z_k}} \tag{8}$$

The input vector $\mathbf{z}$ to $\mathbf{s}(\cdot)$ is sometimes called the "logits", which just means the unscaled output of previous layers. Derive the gradient of $\mathbf{s}$ with respect to the logits, i.e. derive $\frac{\partial \mathbf{s}}{\partial \mathbf{z}}$.

4 Directed Acyclic Graph [10 points]

One important property for a feed-forward network, as discussed in the lectures, is that it must be a directed acyclic graph (DAG). A *DAG is a directed graph that contains no directed cycles*. We will study some of its properties in this question.

Let's define a graph $G = (V, E)$ in which V is the set of all nodes as $\{v_1, v_2, ..., v_i, ...v_n\}$ and E is the set of edges $E = \left\{e_{i,j} = (v_i, v_j) \mid v_i, v_j \in V\right\}$.

A *topological order of a directed graph* $G = (V, E)$ is an ordering of its nodes as $\{v_1, v_2, ..., v_i, ...v_n\}$ so that for every edge (v_i, v_j) we have $i < j$.

There are several lemmas that can be inferred from the definition of a DAG. One lemma is: if G is a DAG, then G has a node with no incoming edges.

4.1 G is a DAG then G has a Topological Ordering [2 points]

Prove that if the graph G is a DAG, then G has a topological ordering.

4.2 G has a Topological Ordering then G is a DAG [2 points]

Prove that if the graph G has a topological order, then G is a DAG.

4.3 Computational Graphs and Backpropagation [6 points]

For the given expression:

$$A = 2 \cdot ((a+b) \cdot (a-b)) \tag{9}$$

We can introduce intermediate variables such that there is a single mathematical operation per assignment such as:

$$t_1 = a+b \qquad t_2 = a-b \qquad t_3 = t_1 \cdot t_2 \qquad A = 2 \cdot t_3 \tag{10}$$

We can then use that to draw the corresponding computation graph based on the introduced variables as shown in Figure 2:

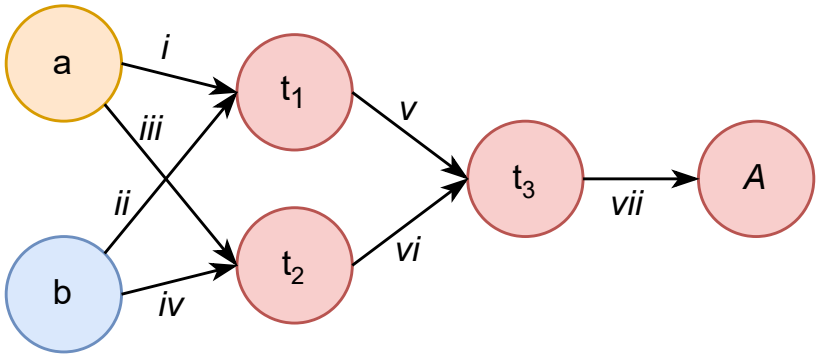


Figure 2: Computation Graph based on the introduced intermediate variables.

Having the computation graph, we can write down the products of partial derivatives that are computed by a node during backpropagation for all of our nodes. Indicating to which node the results are sent to we can write:

$$\begin{aligned} vii &:= \frac{\partial A}{\partial A}\frac{\partial A}{\partial t_3} \quad vi := \frac{\partial A}{\partial t_3}\frac{\partial t_3}{\partial t_2} \quad v := \frac{\partial A}{\partial t_3}\frac{\partial t_3}{\partial t_1} \\ iv &:= \frac{\partial A}{\partial t_2}\frac{\partial t_2}{\partial b} \quad iii := \frac{\partial A}{\partial t_2}\frac{\partial t_2}{\partial a} \quad ii := \frac{\partial A}{\partial t_1}\frac{\partial t_1}{\partial b} \\ i &:= \frac{\partial A}{\partial t_1}\frac{\partial t_1}{\partial a} \end{aligned} \tag{11}$$

With this we can now write down the sum of partial derivative products that calculates the derivative of the output node w.r.t. to this node for all of our nodes as:

$$A: \frac{\partial A}{\partial A} \qquad t_3: \frac{\partial A}{\partial t_3} \qquad t_2: \frac{\partial A}{\partial t_2} \qquad t_1: \frac{\partial A}{\partial t_1}$$

$$b: \frac{\partial A}{\partial t_2}\frac{\partial t_2}{\partial b} + \frac{\partial A}{\partial t_1}\frac{\partial t_1}{\partial b} \qquad a: \frac{\partial A}{\partial t_2}\frac{\partial t_2}{\partial a} + \frac{\partial A}{\partial t_1}\frac{\partial t_1}{\partial a} \tag{12}$$

Now it is your turn. Given is the expressions:

$$A = \frac{e^a}{\sqrt{b}} \log(a + b) \tag{13}$$

a) Introduce intermediate variables (t_1, t_2, t_3, ...) such that there is a single mathematical operation per assignment. Reuse intermediate variables whenever possible.

b) Draw the corresponding computation graph based on the variables you introduced in a).

c) For each edge in your computation graph, write down the products of partial derivatives $\left(\text{e.g. } \frac{\partial \mathcal{L}}{\partial y}\frac{\partial y}{\partial x}\right)$ that are computed by this edge during backpropagation. For each such product indicate to which node the results are sent to (e.g. by writing down the products close to the corresponding graph edges).
Note: you do not need to symbolically calculate or simplify the derivative for specific operations.

d) At each node, write down the sum of partial derivative products that calculates the derivative of the output node w.r.t. to this node. Your expressions only need to contain the direct children for the node and again you do not need to symbolically calculate or simplify the derivatives.

5 Logistic Regression [5 points]

Using the chain rule, show that:

$$\nabla_{\mathbf{w}} \frac{1}{B} \sum_{i=1}^{B} \mathcal{L}(y_i, \hat{y}_i) = \frac{1}{B} \sum_{i=1}^{B} (\hat{y}_i - y_i) \mathbf{x}_i \tag{14}$$

where

$$\begin{aligned} \mathcal{L}(y_i, \hat{y}_i) &= -y \log(\hat{y}) - (1-y) \log(1-\hat{y}) \\ \hat{y}_i &= \sigma(\mathbf{w}^\top \mathbf{x}_i) \end{aligned} \tag{15}$$

Code [60 points + 5 points bonus]

Overview

In this assignment you will first get an introduction to the Numpy package provided by Python. Afterward you will explore how to use the previously learned `Numpy` functions on a real task as well as learn how to create and work with class objects. The third part of the assignment gives you an introduction to the basics of Pytorch tensor operations. Whereas the fourth part gives you an introduction on how to compute derivatives and gradients for these `Pytorch` tensors. In the last two parts, you'll learn what linear regression and logistic regression are all about, including when and for what to use these regression algorithms.

Testing

- For all `Google Colab Notebooks`, we have implemented test cells directly beneath the code snippets you are going to implement:

  ```
  !python ./tests/<test_file> --test_case <name_of_test>
  ```

  You can just run these test cells to test your implementations.

- Besides that many IDEs (e.g., `PyCharm`) offer the option to run unit tests directly.

- Additionally, you can run via the command line by using:

  ```
  $ python -m unittest tests.<name_of_test>
  ```

- To score the whole assignment the way we do, run every test at once from the `Programming` folder:

  ```
  $ bash run_all_tests.sh
  ```

However, passing all local tests neither means your code is free of bugs nor guarantees that you will receive full credits for the coding section. Your code will be tested via additional tests which are not present in your local unit tests.

ToDo

1. Implement the Colab Notebook `numpy_basics.ipynb`.

2. Implement the Colab Notebook `numpy_applied.ipynb`.
3. Implement the Colab Notebook `torch_tensors_basics.ipynb`.
4. Implement the Colab Notebook `derivatives_and_graphs.ipynb`.
5. Implement the Colab Notebook `linear_regression.ipynb`.
6. Implement the Colab Notebook `logistic_regression.ipynb`.

6 Numpy Basics [12 points]

Numpy is the main, fundamental package for scientific computing in Python. It is well-optimized, fast, easy to use and provides lots of useful mathematical functions as well as linear algebra routines. Thanks to its fast and versatile vectorization, indexing and broadcasting concepts it poses the standard for array computing in Python. You will get to know some of the most important functions and learn how to use these which in the end will help you succeed in future assignments.

ToDo

1. Follow the instructions in `numpy_basics.ipynb` to finish this exercise.

6.1 Basic Numpy Functions [12 points]

6.1.1 General Array Properties [0.5 point]

Extract the dimensions of a given Numpy-array.

ToDo

1. Implement the method `extract_array_dimensions_01` which returns the shape of the given array as a tuple.
2. Implement the method `extract_array_dimensions_02` which returns the product of the dimensions of the given array as a scalar.

6.1.2 Creating Arrays [1.5 points]

Create numpy arrays in various ways.

ToDo

1. Implement the method `create_numpy_arrays_01` which creates an array of a specified shape filled with zeros.
2. Implement the method `create_numpy_arrays_02` which creates an array of a specified shape filled with ones.
3. Implement the method `create_numpy_arrays_03` which creates an array with the shape of the given array filled with zeros.
4. Implement the method `create_numpy_arrays_04` which creates an array of values within a specified range and with a specified step size.

5. Implement the method `create_numpy_arrays_05` which creates an array of a specified number of evenly spaced values between two specified values.

6.1.3 Array Handling [1.5 points]

Handle and combine numpy arrays.

ToDo

1. Implement the method `handle_numpy_arrays_01` which reshapes the given array to the specified shape.
2. Implement the method `handle_numpy_arrays_02` which stacks the given arrays vertically.
3. Implement the method `handle_numpy_arrays_03` which stacks the given arrays horizontally.
4. Implement the method `handle_numpy_arrays_04` which concatenates the two given arrays along the specified axis.
5. Implement the method `handle_numpy_arrays_05` which repeats the given array the specified number of times (row-wise and/or column-wise).
6. Implement the method `handle_numpy_arrays_06` which transposes the given array.

6.1.4 Calculating with Arrays [2 points]

Use numpy arrays for calculations.

ToDo

1. Implement the method `calculations_with_numpy_arrays_01` which calculates the element-wise sum of two arrays.
2. Implement the method `calculations_with_numpy_arrays_02` which calculates the dot product of two arrays.
3. Implement the method `calculations_with_numpy_arrays_03` which calculates the element-wise product of two arrays.
4. Implement the method `calculations_with_numpy_arrays_04` which calculates the sum of the elements of an array over a given axis.
5. Implement the method `calculations_with_numpy_arrays_05` which calculates the gradient of an N-dimensional array.

6. Implement the method `calculations_with_numpy_arrays_06` which calculates the cross product of two vectors.
7. Implement the method `calculations_with_numpy_arrays_07` which calculates the element-wise square root of an array.
8. Implement the method `calculations_with_numpy_arrays_08` which returns, as a 1D array, only those values of an array that are smaller than a given threshold (hint: masking).

6.1.5 Statistics with Arrays [1 point]

Calculate statistics on numpy arrays.

ToDo

1. Implement the method `statistics_with_numpy_arrays_01` which returns the minimum value of an array.
2. Implement the method `statistics_with_numpy_arrays_02` which returns the maximum value of an array.
3. Implement the method `statistics_with_numpy_arrays_03` which returns the index of the minimum value of an array.
4. Implement the method `statistics_with_numpy_arrays_04` which returns the index of the maximum value of an array.
5. Implement the method `statistics_with_numpy_arrays_05` which calculates the mean of an array along a specified axis.
6. Implement the method `statistics_with_numpy_arrays_06` which calculates the standard deviation of an array along a specified axis.

6.1.6 Array Slicing 1D [2 points]

Apply smart indexing and slicing to extract specific values from 1D-arrays, like the one presented in Figure 3.



Figure 3: Example for a 1D numpy array.

This shall only represent an example. Your code will be tested on different arrays, too.

ToDo

1. Implement the method `slicing_of_numpy_arrays_1d_01` which extracts the first element of the given array.



2. Implement the method `slicing_of_numpy_arrays_1d_02` which extracts the last element of the given array.



3. Implement the method `slicing_of_numpy_arrays_1d_03` which extracts the first half of elements of the given array (if the length is odd, this shall contain the smaller fraction).



4. Implement the method `slicing_of_numpy_arrays_1d_04` which extracts the second half of elements of the given array (if the length is odd, this shall contain the larger fraction).



5. Implement the method `slicing_of_numpy_arrays_1d_05` which extracts the following pattern of elements:



6. Implement the method `slicing_of_numpy_arrays_1d_06` which extracts the following pattern of elements:



7. Implement the method `slicing_of_numpy_arrays_1d_07` which extracts the following pattern of elements:



6.1.7 Array Slicing 2D [2.5 points]

Apply smart indexing and slicing to extract specific values from 2D-arrays, like the one presented in Figure 4.

-1	15	4	6
5	-4	17	0
3	3	-8	6
7	-1	-10	2

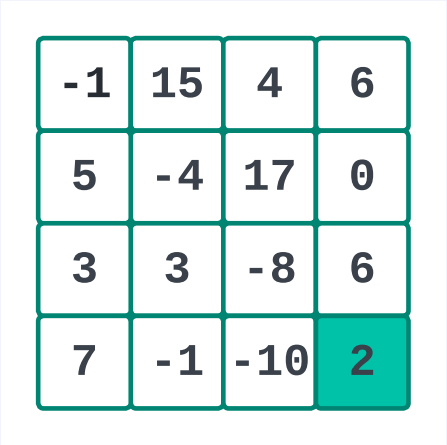
Figure 4: Example for a 2D numpy array.

ToDo

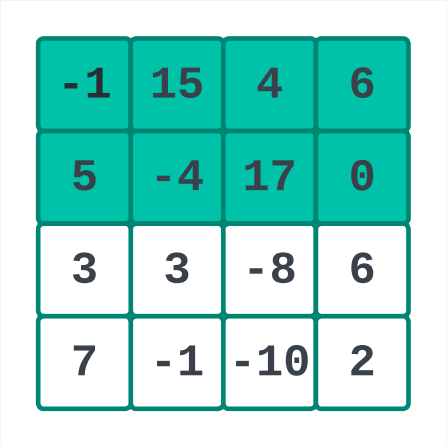
1. Implement the method `slicing_of_numpy_arrays_2d_01` which extracts the first element of the given array.

-1	15	4	6
5	-4	17	0
3	3	-8	6
7	-1	-10	2

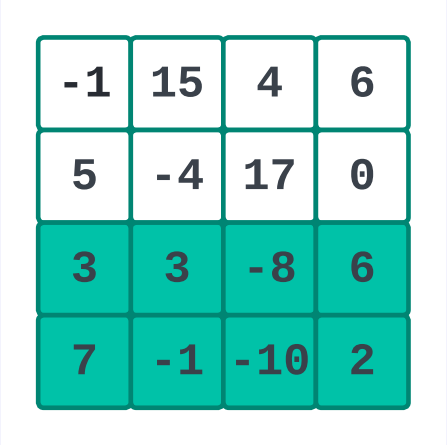
2. Implement the method `slicing_of_numpy_arrays_2d_02` which extracts the last element of the given array.



3. Implement the method `slicing_of_numpy_arrays_2d_03` which extracts the row-wise first half of elements of the given array (if the length is odd, this shall contain the smaller fraction).



4. Implement the method `slicing_of_numpy_arrays_2d_04` which extracts the row-wise second half of elements of the given array (if the length is odd, this shall contain the larger fraction).



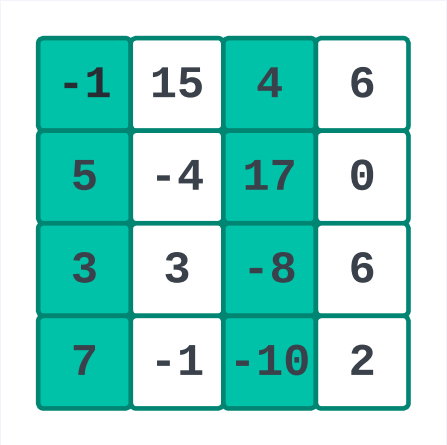
5. Implement the method `slicing_of_numpy_arrays_2d_05` which extracts the column-wise first half of elements of the given array (if the length is odd, this shall contain the smaller fraction).

-1	15	4	6
5	-4	17	0
3	3	-8	6
7	-1	-10	2

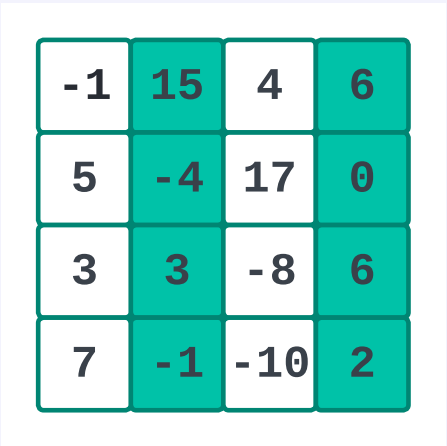
6. Implement the method `slicing_of_numpy_arrays_2d_06` which extracts the column-wise second half of elements of the given array (if the length is odd, this shall contain the larger fraction).

-1	15	4	6
5	-4	17	0
3	3	-8	6
7	-1	-10	2

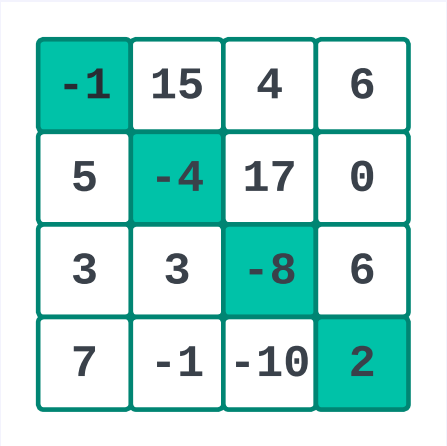
7. Implement the method `slicing_of_numpy_arrays_2d_07` which extracts every element at an odd column in row-wise order of the given array.



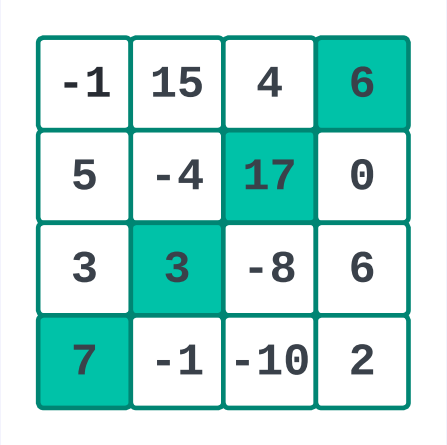
8. Implement the method `slicing_of_numpy_arrays_2d_08` which extracts every element at an even column in row-wise order of the given array.



9. Implement the method `slicing_of_numpy_arrays_2d_09` which extracts the diagonal elements in row-wise order of the given array (only symmetric 2D arrays will be considered).



10. Implement the method `slicing_of_numpy_arrays_2d_10` which extracts the reverse diagonal elements in row-wise order of the given array (only symmetric 2D arrays will be considered).



6.1.8 More Advanced Operations with Arrays [1 point]

Apply more complex operations on numpy arrays, such as sorting by a given column or using Numpy's broadcasting rule during element-wise operations.

ToDo

1. Implement the method `sort_array` which sorts the given array by the specified column.
2. Implement the method `multiply_arrays_with_broadcasting` which multiplies the two given 1D arrays with each other and the two given n-dimensional arrays with each other, using broadcasting.

7 Numpy Applied [11 points + 5 points bonus]

In this exercise you will apply the previously learned basics on real tasks.

ToDo

1. Follow the instructions in `numpy_applied.ipynb` to finish this exercise.

7.1 Simple Array Border Pattern [1 point]

In this exercise you will learn how to implement simple patterns with numpy arrays via an example of creating borders to a 2D-array.

Parameters:

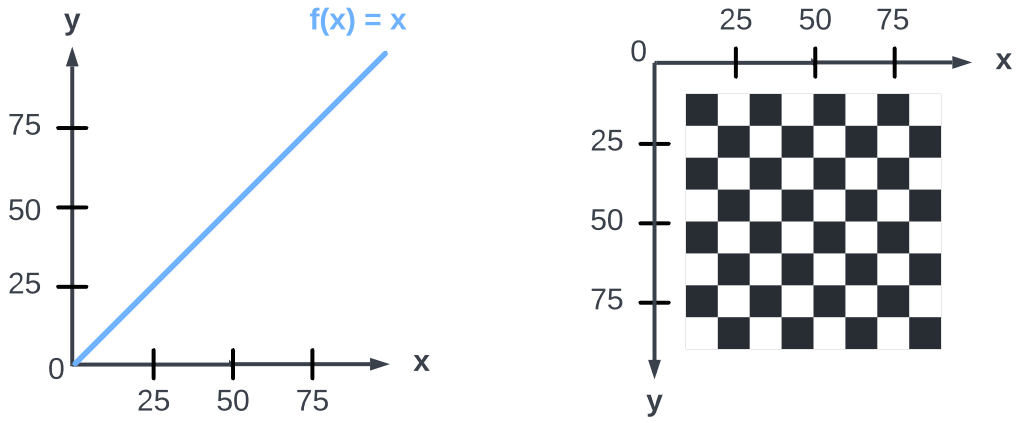
- `arr_shape:` Defines the shape of the 2D-array (tuple).

ToDo

1. Implement the method `create_array_border` which creates a 2D array of the given shape with 1 on the border and 0 inside, and then adds a border filled with 0's around it.

7.2 Checkerboard Pattern [3 points]

In this first exercise you will be implementing a checkerboard pattern in the class `CheckerBoard` with an adaptable `size` and `tile_size`. An example of how the result shall look like can be seen in Figure 5:



(a) Maths coordinate system

(b) Image coordinate system

Figure 6: Visualization of the origin and axes differences of the two coordinate systems.

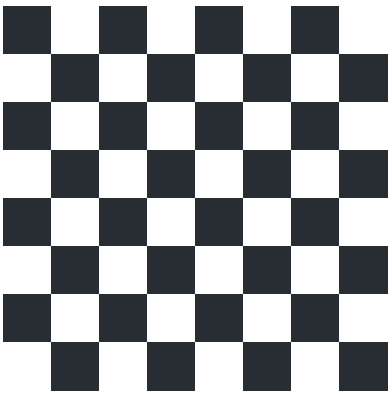


Figure 5: Example for the checkerboard pattern.

Please note that if we are talking about images, the coordinate system differs slightly from what you are used to from other (math) classes. In maths, if we talk about 2D functions within the Cartesian Coordinate System, the origin of the coordinate system is in the lower left corner with the x-axis ranging from left to right and the y-axis ranging from bottom to top.
However, for images the origin of the coordinate system is in the upper left corner instead with the x-axis ranging from left to right and the y-axis ranging from top to bottom. Furthermore, for images, a particular pair of (x-coordinate, y-coordinate) describes a single pixel within the image.
As a result, this means that when you have to deal with x- and y-coordinates of images, you have to treat them differently when dealing with x- and y-coordinates for functions. See figure 6 for reference.

Hints:

1. `__init__` represents the constructor of the class.
2. Following functions might be useful: `np.zeros`, `np.ones`, `np.tile`, `np.arange`, `np.concatenate`, `np.expand_dims`.
3. To simplify things we assume that the `size` is divisible by the `tile_size` without remainder.
4. Think about how the relation between `size` and `tile_size` has to look like to create a valid checkerboard pattern.

Attention

If you are using the dark mode of your development environment instead of the standard light one, it will most likely happen that the appearance of the colors white and black are completely switched. This means everything written in black is shown in white and vice versa. This affects the appearance of text but might as well affect the appearance of plots you will generate throughout this exercise. This means a figure which should be rendered in the color black will be rendered in the color white and vice versa. However, this accounts only for figures you will create by running parts of the code below. Figures that were provided by us will still appear the way they are, regardless which appearance mode you are using within your development environment. Please keep that in mind if you want to use your development environment's dark mode!

7.2.1 Initialize the Checkerboard Pattern [0.5 point]

Parameters:

- `size:` Defines the width and height of our checkerboard in pixels (int).
- `tile_size:` Defines the width and height of each individual tile in pixels (int).

ToDo

1. Implement the method `__init__` (the constructor) of the class `CheckerBoard`.
2. Store each of the given parameters in an instance variable of class `CheckerBoard`.
3. Create an additional instance variable with the name `pattern` that will store the final checkerboard pattern.

7.2.2 Create the Checkerboard Pattern [2 points]

Parameters: —

ToDo

1. Implement the method `draw` of the class `CheckerBoard` which constructs the checkerboard pattern as a numpy array.
2. Black tiles shall have the value 0 and white tiles shall have the value 1.
3. Make sure that the tile in the top left is colored black.
4. To avoid truncated patterns, make sure your code only allows values for `size` that are evenly dividable by `2*tile_size`.
5. Store the created pattern in the variable `pattern`.

7.2.3 Visualize the Checkerboard Pattern [0.5 point]

Parameters: —

ToDo

1. Implement the method `show` of the class `CheckerBoard` which shows the checkerboard pattern as an image.
2. Display the checkerboard pattern as a grayscale image by using the imshow function of Matplotlib with the respective parameter.

7.3 Shapes [7 points]

In this second exercise you will be implementing a binary pattern consisting of a white circle, and white rectangle in front of a black background. An example of how the result can look like can be seen in Figure 7:



Figure 7: Example for the shapes pattern.

,

Hints:

1. For code-generated images and plots the axes' orientations differ from the ones you may know from math classes. Here, we use the pixel coordinate system. Conventionally the coordinate (0, 0) is located in the upper left corner of the image, with the x-axis ranging from left to right (along the columns of the image) and the y-axis ranging from top to bottom (along the rows of the image).

2. For the rectangle shape, take into account that the given x1 and y1 coordinates could be smaller than the respective x2 and y2 coordinates.

3. For the circular shape, think of a formula describing the circle with respect to pixel coordinates.

Attention

If you are using the dark mode of your development environment instead of the standard light one, it will most likely happen that the appearance of the colors white and black are completely switched. This means everything written in black is shown in white and vice versa. This affects the appearance of text but might as well affect the appearance of plots you will generate throughout this exercise. This means a figure which should be rendered in the color black will be rendered in the color white and vice versa. However, this accounts only for figures you will create by running parts of the code below. Figures that were provided by us will still appear the way they are, regardless which appearance mode you are using within your development environment. Please keep that in mind if you

want to use your development environment's dark mode!

7.3.1 Create the Rectangle Shape [2 points]

As a first step we will be implementing the class `Rectangle` which creates a white rectangle in front of a black background. An example of how the result can look like can be seen in Figure 8:



Figure 8: Example for the rectangle pattern.

Parameters:

- `size:` Defines the width and height of our pattern in pixels (int).
- `x1y1:` Defines the x- and y-coordinate in pixels of one corner of the rectangle (tuple).
- `x2y2:` Defines the x- and y-coordinate in pixels of the opposing corner to the x1y1-specified corner of the rectangle (tuple).

ToDo

1. Implement the method `__init__` (the constructor) of the class `Rectangle`.
2. Implement the method `draw` of the class `Rectangle` which constructs the pattern as a numpy array.
3. Implement the method `show` of the class `Rectangle` which plots the pattern.

Hints:

1. Take into account that the given x1 and y1 coordinates could be smaller than the respective x2 and y2 coordinates.

7.3.2 Create the Circle Shape [3 points]

As the second step we will be implementing the class `Circle` which creates a white circle in front of a black background. An example of how the result can look like can be seen in Figure 9:



Figure 9: Example for the circle pattern.

Parameters:

- `size:` Defines the width and height of our pattern in pixels (int).
- `radius:` Defines the radius of the circle in pixels (int).
- `center:` Defines the x- and y-coordinate of the circle center in pixels (int).

ToDo

1. Implement the method `__init__` (the constructor) of the class `Circle`.
2. Implement the method `draw` of the class `Circle` which constructs the pattern as a numpy array.
3. Implement the method `show` of the class `Circle` which plots the pattern.

Hints:

1. Think of a formula describing the circle with respect to pixel coordinates.

7.3.3 Combine and Plot the Shapes [2 points]

As the last step we will call the previously created classes `Rectangle` and `Circle` to create the wanted shape pattern.

Parameters:

- `size:` Defines the width and height of our pattern in pixels (int).
- `x1y1:` Defines the x- and y-coordinate in pixels of one corner of the rectangle (tuple).
- `x2y2:` Defines the x- and y-coordinate in pixels of the opposing corner to the x1y1-specified corner of the rectangle (tuple).
- `radius:` Defines the radius of the circle in pixels (int).
- `center:` Defines the x- and y-coordinate of the circle center in pixels (int).

ToDo

1. Implement the method `combine_and_plot_shapes` which creates the individual patterns of `Rectangle` and `Circle`, combines them in one image and plots it as a gray-scale image.

Hints:

1. Think about how to process overlapping shapes in one image to keep the binary style of the pattern.

7.4 Bonus: Color Spectrum [5 points]

In this third bonus exercise you will be implementing an RGB color spectrum in the class `Spectrum`. Since this exercise is optional it won't count towards your grade on this assignment. However, it might be quite interesting and fun to code, so feel free to do it! An example of how the result should look like can be seen in Figure 10:

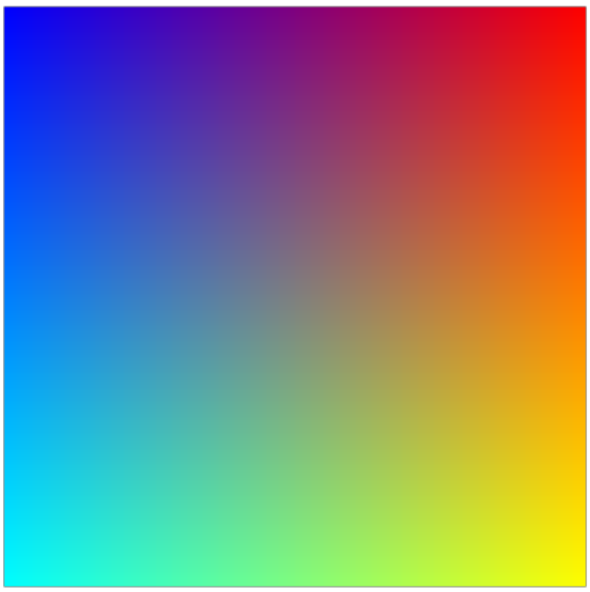


Figure 10: Example for the spectrum pattern.

Parameters:

- `size:` Defines the width and height of our pattern in pixels (int).

ToDo

1. Implement the method `__init__` (the constructor) of the class `Spectrum`.
2. Implement the method `draw` of the class `Spectrum` which constructs the pattern as a numpy array.
3. Implement the method `show` of the class `Spectrum` which plots the pattern.

Hints:

1. RGB images have 3 channels.
2. A spectrum consists of rising values across a specific dimension.
3. The intensity minimum and maximum should be 0.0 and 1.0, respectively, for each of the three color channels.

4. Check out the corners of the reference image to figure out the underlying distribution of channels.

8 Torch Tensors Basics [11.5 points]

This third part of the exercise gives you an introduction to the basics of tensor operations. Tensors are an essential part of PyTorch which we will use later in the course to built our neural networks. Hence, understanding the fundamentals now will help you succeed in later assignments.

ToDo

1. Follow the instructions in `torch_tensors_basics.ipynb` to finish this exercise.

8.1 Install Torch [0 point]

Please note that the provided `environment.yaml` file does not contain a PyTorch Installation for you. This is due to the fact that you may run the code on a `CPU` or `GPU`, or for a different `CUDA version`. Hence, as a first step, please refer to the official documentation of `PyTorch` to install the version that is suited best for you and your equipment.

8.2 Tensor Basics [4 points]

`PyTorch` is an open source machine learning framework which is used for applications such as computer vision and natural language processing. It is a free and open-source software which provides the following two high-level features:

- Tensor computing (like NumPy) with strong acceleration via graphics processing units (GPU)
- Deep neural networks built on a tape-based automatic differentiation system

In this exercise you will learn some basic functions the torch package provides you with to create, define and index tensors.

8.2.1 Create a Tensor [1.75 points]

In this exercise you will learn different ways of how to create a tensor.

Parameters:

- `shape:` Defines the shape of the tensor (tuple).
- `lst:` Defines an n-dimensional list of values (list).
- `a:` Defines an n-dimensional numpy array of values (np.ndarray).

ToDo

1. Implement the method `create_tensor_with_specified_shape` which creates a tensor with a specified `shape`.
2. Implement the method `create_tensor_from_list` which creates a tensor from a given list `lst`.
3. Implement the method `create_tensor_from_numpy_array` which creates a tensor from a given numpy array `a`.
4. Implement the method `create_random_tensor` which creates a tensor of a specified shape `shape` with random values.
5. Implement the method `create_tensor_filled_with_zeros` which creates a tensor of a specified shape `shape` filled with zeros.
6. Implement the method `create_tensor_filled_with_ones` which creates a tensor of a specified shape `shape` filled with ones.
7. Implement the method `create_tensor_with_the_diagonal_filled_with_ones` which creates a 2D-tensor with a specified number of rows and columns `shape` for which its diagonal is filled with ones.

8.2.2 Extract Tensor Information [1.25 point]

In this exercise you will learn how to extract information from an existing tensor.

Parameters:

- `t:` Defines the tensor (torch.tensor).

ToDo

1. Implement the method `extract_the_type_of_a_tensor` which extracts the type of a given tensor `t`.
2. Implement the method `extract_the_data_type_of_a_tensor` which ex-

tracts the data type of a given tensor `t`.

3. Implement the method `extract_the_dimensions_of_a_tensor` which extracts the dimension of a given tensor `t`.
4. Implement the method `extract_the_shape_of_a_tensor` which extracts the shape of a given tensor `t`.
5. Implement the method `extract_the_number_of_elements_of_a_tensor` which extracts the number of elements of a given tensor `t`.

8.2.3 Indexing and Slicing [1 point]

In this exercise you will learn how to access the different elements of a tensor. For this task, we will consider the following two-dimensional tensor which can be seen in Figure 11:

11	12	13
21	22	23
31	32	33

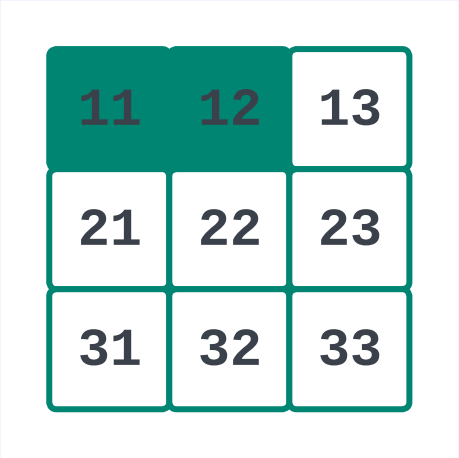
Figure 11: Used 2D tensor.

Parameters:

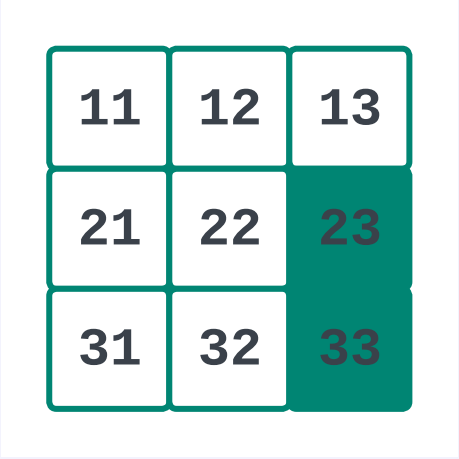
- `t:` Defines the tensor (torch.tensor).

ToDo

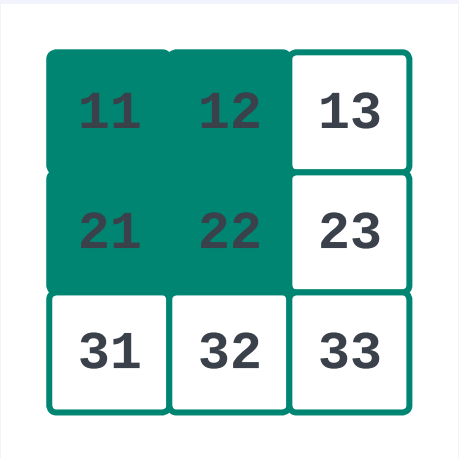
1. Implement the method `slicing_01` to extract the following slice of the given tensor `t`:



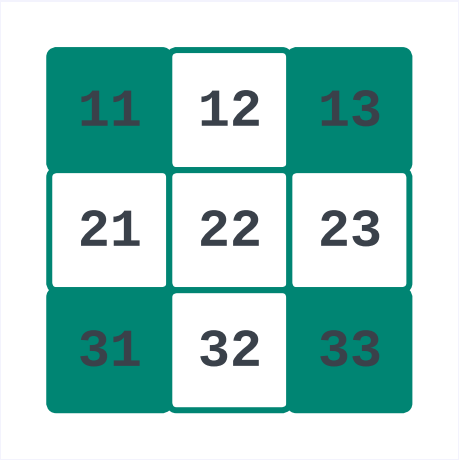
2. Implement the method `slicing_02` to extract the following slice of the given tensor `t`:



3. Implement the method `slicing_03` to extract the following slice of the given tensor `t`:



4. Implement the method `slicing_04` to extract the following slice of the given tensor `t`:



8.3 Tensor Calculations [2.5 points]

We can also do calculations with tensors. In this exercise you will learn some of the basic calculations that can be done with tensors.

8.3.1 Tensor Addition [0.25 point]

In this exercise you will learn how to add two tensors together.

Parameters:

- `t1:` Defines the first tensor (torch.tensor).
- `t2:` Defines the second tensor (torch.tensor).

ToDo

1. Implement the method `tensor_summation` which calculates the sum of two given tensors `t1` and `t2`.

8.3.2 Scalar Multiplication [0.25 point]

In this exercise you will learn how to multiply a scalar with a tensor.

Parameters:

- `t:` Defines a tensor (torch.tensor).
- `s:` Defines a scalar (float).

ToDo

1. Implement the method `scalar_multiplication` which multiplies a given tensor `t` with a given scalar `s`.

8.3.3 Hadamard Product [0.25 point]

In this exercise you will learn how to calculate the hadamard product of two tensors by multiplying them element-wise.

Parameters:

- `t1:` Defines the first tensor (torch.tensor).
- `t2:` Defines the second tensor (torch.tensor).

ToDo

1. Implement the method `hadamard_product` which calculates the hadamard (element-wise) product of two given tensors `t1` and `t2`.

8.3.4 Matrix Multiplication [0.25 point]

In this exercise you will learn how to calculate the matrix product of two tensors.

Parameters:

- `t1:` Defines the first tensor (torch.tensor).
- `t2:` Defines the second tensor (torch.tensor).

ToDo

1. Implement the method `matrix_product` which calculates the matrix product of two given tensors `t1` and `t2`.

8.3.5 Dot Product [0.25 point]

In this exercise you will learn how to calculate the dot product of two tensors.

Parameters:

- `t1:` Defines the first tensor (torch.tensor).
- `t2:` Defines the second tensor (torch.tensor).

ToDo

1. Implement the method `dot_product` which calculates the dot product of two given 1D tensors `t1` and `t2`.

8.3.6 Square Tensor [0.25 point]

In this exercise you will learn how to square all elements in a tensor.

Parameters:

- `t:` Defines a tensor (torch.tensor).

ToDo

1. Implement the method `square_tensor` which squares all elements of a given tensor `t`.

8.3.7 Tensor Broadcasting [0.5 point]

In this exercise you will learn how to take advantage of tensor broadcasting during basic arithmetic operations.

Parameters:

- `t:` Defines a tensor (torch.tensor).
- `value:` Defines the integer value to fill the tensor with.

ToDo

1. Implement the method `fill_tensor_with_value` which fills a given tensor `t` with a specific value using broadcasting and basic arithmetic operations.

Hints:

1. Try to only use +-/* operators. Don't create new tensor with torch.tensor or use any torch functions.

8.3.8 Matrix Multiplication with Broadcasting [0.5 point]

In this exercise you will learn how to perform a matrix multiplication that may require broadcasting.

In a classroom quiz, students were evaluated based on their answers to a series of questions. Given a 2D list of student answers, where each row corresponds to an individual student's set of answers to these questions, and a 1D list of question points indicating the points awarded for each respective question, compute the aggregate score for every student and find the student with the highest score. The student answers are either 0 or 1, with 1 meaning the answer is correct.

Parameters:

- `student_answers:` Defines the 2D tensor where each row corresponds to an individual student's set of answers (torch.tensor).
- `question_points:` Defines the 1D tensor indicating the points awarded for each respective question (torch.tensor).

ToDo

1. Implement the method `find_top_scoring_student` which computes the aggregate score for every student and returns the index (presented within a tensor) of the student who achieved the topmost score. The smallest index is returned in case of a tie.

For example, the expected output for the input situation visualized in Figure 12 is 2 because the third student (2-indexed) has the highest aggregate score.

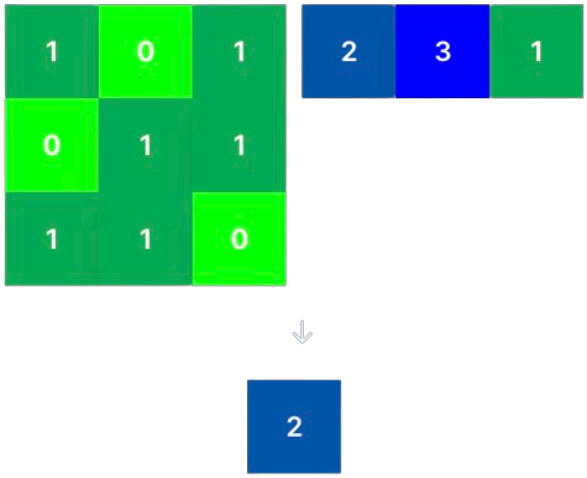


Figure 12: Example for the input student answers array (left) and question points array (right) and the expected output.

8.4 Tensor Statistics [1.5 points]

We can also do statistics with tensors. In this exercise you will learn some of the basic commands.

8.4.1 Tensor Min [0.25 point]

In this exercise you will learn how to find the minimum value in a tensor.

Parameters:

- `t:` Defines a tensor (torch.tensor).

> **ToDo**
>
> 1. Implement the method `minimum` which finds the minimum value within a tensor `t` and returns it as another tensor.

8.4.2 Tensor Max [0.25 point]

In this exercise you will learn how to find the maximum value in a tensor.

Parameters:

- `t:` Defines a tensor (torch.tensor).

> **ToDo**
>
> 1. Implement the method `maximum` which finds the maximum value within a tensor `t` and returns it as another tensor.

8.4.3 Tensor Mean [0.25 point]

In this exercise you will learn how to find the mean value of a tensor.

Parameters:

- `t:` Defines a tensor (torch.tensor).

ToDo

1. Implement the method `mean` which finds the mean value of a tensor `t` and returns it as another tensor.

Hints:

1. You may need to cast the given tensor to another datatype before the mean can be calculated...

8.4.4 Tensor Standard Deviation [0.25 point]

In this exercise you will learn how to find the standard deviation of a tensor.

Parameters:

- `t:` Defines a tensor (torch.tensor).

ToDo

1. Implement the method `standard_deviation` which finds the standard deviation of a tensor `t` and returns it as another tensor.

Hints:

1. You may need to cast the given tensor to another datatype before the standard deviation can be calculated...

8.4.5 Tensor Statistics Applied [0.5 point]

In this exercise you will learn how to apply the basic statistics commands to find the row with the highest mean value in a tensor.

Parameters:

- `t:` Defines the tensor (torch.tensor).

ToDo

1. Implement the method `find_row_with_highest_mean` which finds the index of the row with the highest mean value in a tensor `t` and returns it as an integer.

For example, the expected output for the input tensor in Figure 13 is 2 because the third row (2-indexed) has the highest mean.

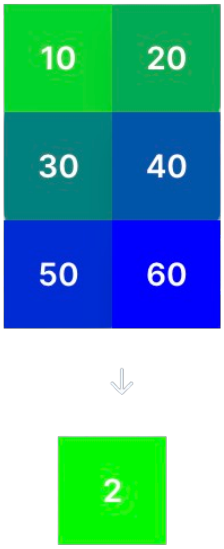


Figure 13: Example for the input tensor and expected output.

Hints:

1. Use torch.argmax to find the index of the highest score.

8.5 Tensor Operations [3.5 points]

We can further do additional operations with tensors which are needed to avoid shape or type errors during calculations. In this exercise you will learn some of the most important commands for tensors.

8.5.1 Casting [0.75 point]

In this exercise you will learn how to apply casting on tensors.

Parameters:

- `t:` Defines a tensor (torch.tensor).

ToDo

1. Implement the method `cast_to_list` which casts a tensor `t` to a python list having the same dimensions.
2. Implement the method `cast_to_numpy` which casts a tensor `t` to a numpy array having the same dimensions.
3. Implement the method `cast_to_double` which casts a tensor `t` containing integer values to a tensor containing double values.

8.5.2 Transform a Tensor [1.75 points]

In this exercise you will learn how to transform a tensor to a different shape or a different arrangement of its elements. These transformations can be necessary to allow the combination of tensors or to enable the calculations within the neural networks we will be working with later on in the course.

Parameters:

- `t:` Defines a tensor (torch.tensor).
- `shape:` Defines the new shape (tuple).

ToDo

1. Implement the method `reshape_contiguous_tensor` which reshapes a contiguous tensor `t` to a new shape `shape`.
2. Implement the method `reshape_non_contiguous_tensor` which reshapes a non-contiguous tensor `t` to a new shape `shape`.
3. Implement the method `swap` which swaps the first axis (dimension) of a 3D-tensor `t` with its last axis (dimension).
4. Implement the method `trans` which transposes a 2D-tensor `t`.
5. Implement the method `pave` which paves an nD-tensor `t` to a 1D tensor.

8.5.3 Concatenate Tensors [1 point]

In this exercise you will learn how to concatenate tensors together to one single tensor.

Parameters:

- `t1:` Defines the first tensor (torch.tensor).
- `t2:` Defines the second tensor (torch.tensor).
- `ax:` Defines the concatenation axis (int).

ToDo

1. Implement the method `concat` which concatenates two tensors `t1` and `t2` along the given axis `ax`.

Hints:

1. Some tensor combinations can not be concatenated together directly but require some transformations first (e.g transposing the second tensor to match the shape of the first one).
2. Some tensor combinations can never be concatenated together and should be dealt with accordingly (e.g. raise an error).

9 Derivatives and Graphs [5.5 points]

This fourth and last part of the assignment gives you an introduction on how to compute derivatives and gradients for Pytorch tensors. When applying deep neural networks, one always has to compute gradients of multiple different network parameters. Hence, understanding the fundamentals now will help you succeed in later assignments.

ToDo

1. Follow the instructions in `derivatives_and_graphs.ipynb` to finish this exercise.

9.1 Plotting [0 point]

We provide you with a function that will allow you to visualize the functions and their derivatives of later tasks. You don't have to code anything here.

ToDo

1. Nothing

9.2 Derivatives [3.5 points]

In this exercise you will learn how to calculate the gradient (derivative) of basic functions which do calculations based on Pytorch tensors.

9.2.1 Derivative of a Simple Function at a Specific Position [1 point]

In this exercise you will learn how to calculate the derivative of the simple function $f(x)$ given as:

$$f(x) = x^2$$

at position $x = 2$.

Remember the derivative $f'(x)$ of the function $f(x)$ with respect to x is calculated as:

$$f'(x) = \frac{df(x)}{dx} = (x^2)' = 2x$$

which means that $f'(x)$ for $x = 2$ is:

$$f'(x = 2) = 4$$

Of course, we could hard code all of that and compute our derivative. However, we would do it only for a single function at a single evaluation position. This means that we would need to code everything again for a different function or a different position we want our derivative to be evaluated at. Hence, this is not quite efficient.

But since each one of you is a highly skilled programmer and probably wants to stay one, you want to use a more convenient and efficient way to compute derivatives of different functions. And for that Pytorch will give you the answer. In the next few steps, we will learn how to use a few simple commands provided by Pytorch to achieve that goal.

ToDo

1. Implement the method `derivative_of_simple_function_at_position` which calculates the derivative of $f(x)$ at $x = 2$.

Hints:

1. Check out the requires_grad option for Pytorch tensors.
2. Check out the function backward() provided by Pytorch for tensors.

9.2.2 Derivative of a More Complicated Function at a Specific Position [1 point]

Let's try what we have learned above for a more complicated function $g(x)$. The function $g(x)$ is given as:

$$g(x) = 2x^3 + x^2 + x$$

And we want to calculate its derivative $g'(x)$ at position $x = 5$.

ToDo

1. Implement the method `derivative_of_more_complicated_function_at_position` which calculates the derivative $g'(x)$ of $g(x)$ at $x = 5$

9.2.3 Derivative of an Entire Function [1.5 points]

Having the derivative at a specific position $x^\star$ is cool, but in general we are interested in the derivative of an entire function. Calculation of the latter can also be done quite fast by using the autograd functionality of Pytorch. Let's take a look at the Rectified Linear Unit (ReLU) function which is given as:

$$f(x) = max(0, x)$$

We want to calculate the derivative of the entire function at every point of x.

ToDo

1. Implement the method `derivative_of_relu` which calculates the derivative $f'(x)$ of $f(x)$ at every point in x.

Hints:

1. Check out the sum()-trick provided by Pytorch to get the derivative of a vector valued function.

9.3 Partial Derivatives [2 points]

In this exercise you will learn how to calculate partial derivatives of functions which do calculations based on Pytorch tensors. Consider the function $f(x)$ given as:

$$f(u, v) = v * u + u^2 + 3 * v$$

We want to calculate the derivatives f_u and f_v of $f(u, v)$ with respect to u and v respectively.

ToDo

1. Implement the method `partial_derivatives` which calculates the derivatives f_u and f_v of $f(u, v)$ with respect to u and v respectively.

10 Linear Regression [7 points]

Linear regression (analysis) is the simplest model form to regress a scalar dependent variable y based on a number of one (simple linear regression) or more (multiple linear regression) explanatory variables $X = \{x_1, ...\}$. Due to its simplicity and the elegance that comes with it, it is very popular for various applications in business and academic study. In particular, for situations where you have only a small, limited amount of samples or weak signals, linear regression can still provide a decent approximation to the underlying regression function.
Hence, one can say that linear regression is used to estimate unknown model parameters of so-called linear models by using linear predictor functions to model the data. To do that it focuses on the conditional probability distribution of a variable y given some variables $X = \{x_1, ...\}$, like all other types of regression algorithms. Models that depend linear on their unknown parameters are easier to fit than those which have a non-linear relation. This is due to the fact that the statistical properties of the resulting estimators are easier to determine due to their decreased complexity. In other words linear regression can be used to fit a hyperplane (for the 2D case this is a line) to a set of points by minimizing the distances of these points to that hyperplane.
This can be used to predict for example the trend in stock, or housing prices based on last observations. Figure 14 presents an example of a simple linear regression of some data points $X = \{x_1, ..., x_n\}$.

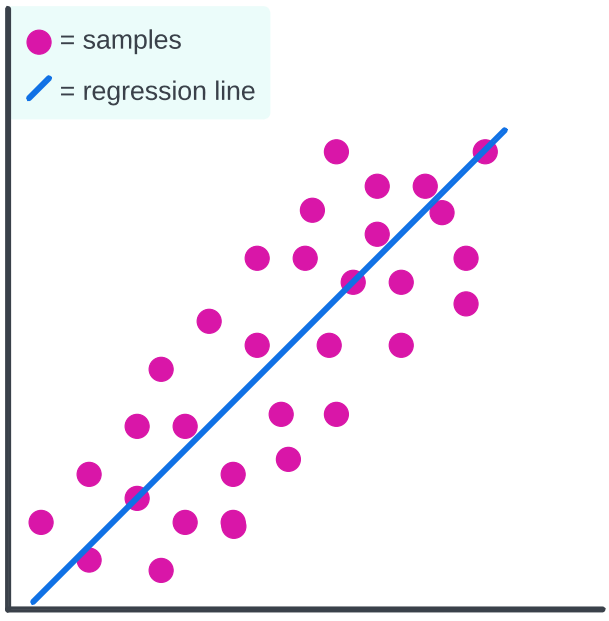


Figure 14: Example of a simple linear regression.

In the lecture you have already seen how to apply linear regression in an analytical way. It is easy, fast and reliable. Or so it seems. Like any other analytical calculation the analytic computation of linear regression can cause you trouble. It works quite

well for a small number of samples/measurements. However, with an increasing amount of samples, calculating the inverse will get more and more expensive until it won't be feasible to calculate it anymore. One possibility to solve this issue is to use the Moore-Penrose Pseudoinverse instead of the "normal" inverse matrix which can be found accurately with singular value decomposition (SVD). In this exercise I want to show you yet another possibility to avoid expensive computations, which is namely the iterative approach. Hence, for this linear regression exercise we will fit our regression line iteratively to our dataset.

ToDo

1. Follow the instructions in `linear_regression.ipynb` to finish this exercise.

10.1 Visualization [0 point]

We provide you with a function that will be used for visualization. We provide this to you, so you can plot your results. You don't have to code anything here.

ToDo

1. Nothing

10.2 Data Loading [1 point]

The first task of this exercise is to load the provided dataset, which are just some random 2D-samples in this case.

ToDo

1. Implement the class `DataLoader`.
2. Implement the constructor `__init__` which receives the following parameters:
   - `file_path:` the path to the file containing the 2D-sample as a string.
3. Implement the method `get_samples` which loads and returns the samples as two separate lists containing their x- and y-values respectively.

10.3 Iterative Linear Regression [3 points]

After we have loaded the dataset correctly, we can start coding our linear regression algorithm. The second task of this exercise is to iteratively regress a linear line to the given data samples.

10.3.1 Loss Function of Linear Regression [0.5 point]

If we want to iteratively regress a line to a set of samples, we first need a metric to measure how well our current regression line actually fits the data. For that we use a so-called **loss function** ($\mathcal{L}$). For the example in Figure 15, the loss in (a) should be smaller than the loss in (b), since the regression line in (a) fits the data way better than the one in (b).

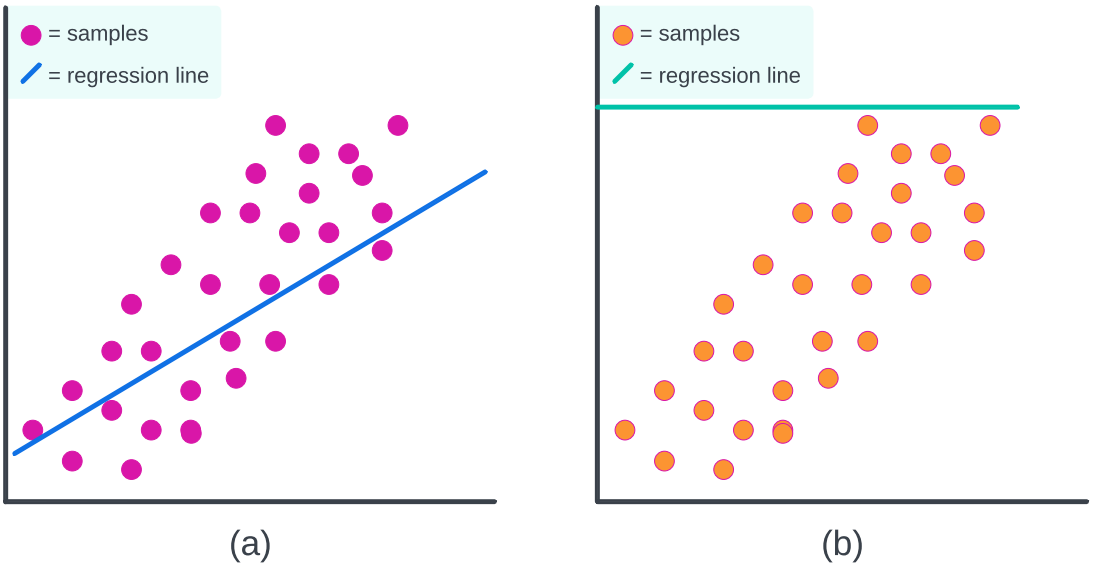


Figure 15: Example of two different regression lines.

For a linear regression model with a scalar input and a scalar output, the loss function is given by:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(w_0 + w_1 x_i - y_i)^2 \tag{16}$$

In this simple 1D case, we are fitting a straight line to the dataset. This line is determined by its offset along the y-axis ($\mathbf{w_0}$) and its slope ($\mathbf{w_1}$).

> **ToDo**

1. Implement the method `loss_linear_regression` which calculates the loss of our linear regression model given the following parameters:
   - `x:` x-values of the samples as a numpy array.
   - `y:` y-values of the samples as a numpy array.
   - `w0:` y-offset of the current regression line.
   - `w1:` slope of the current regression line.

10.3.2 Derivatives of Loss Function of Linear Regression [1 point]

Using the loss function we are able to say how well our current regression line fits the underlying data. Now we want to use that information to iteratively update our two model parameters w_0 (y-offset) and w_1 (slope) to fit the regression line better to the data with each iteration, as it is presented in the Figure 16.

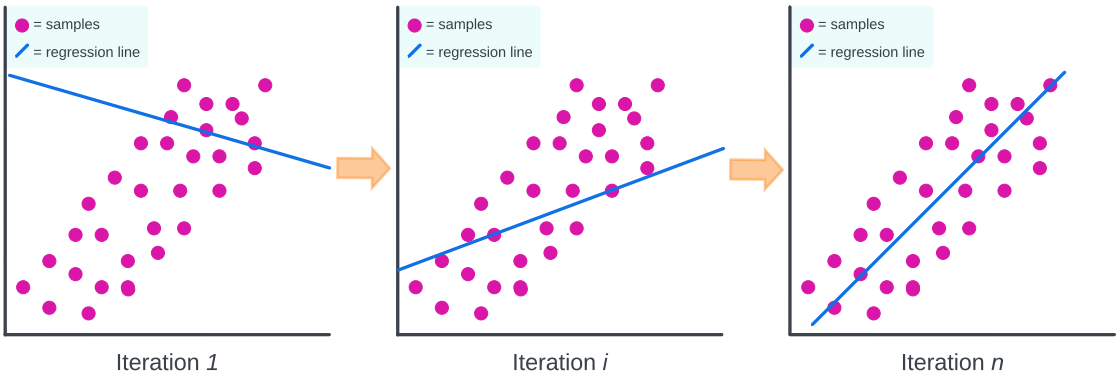


Figure 16: Fit the regression line iteratively to the data.

For that we will update our model parameters with the help of the derivatives of the loss function. Hence, as a next step we will calculate the partial derivative of the loss w.r.t the y-offset $\frac{\partial \mathcal{L}}{\partial w_0}$ as well as the partial derivative of the loss w.r.t the slope $\frac{\partial \mathcal{L}}{\partial w_1}$.

ToDo

1. Implement the method `compute_derivatives_of_loss` which calculates the partial derivatives of the loss function w.r.t w0 and w1 given the following parameters:
   - `x:` x-values of the samples as a numpy array.
   - `y:` y-values of the samples as a numpy array.

- `w0:` y-offset of the current regression line.
- `w1:` slope of the current regression line.

10.3.3 Optimize the Model Parameters [0.5 point]

Since we now know how to calculate the derivative of the loss function $\mathcal{L}$ w.r.t. to w_0 and w_1, we can use that information to iteratively update exactly those model parameters w_0 and w_1 to optimally fit the regression line to our data set. An easy optimization scheme for this is gradient descent (GD). GD is a first-order iterative optimization algorithm to find a local minimum of a differentiable function by repeatedly taking steps in the opposite direction of the gradient of that function at the current point (direction of the steepest descent). An example for this process can be seen in Figure 17.

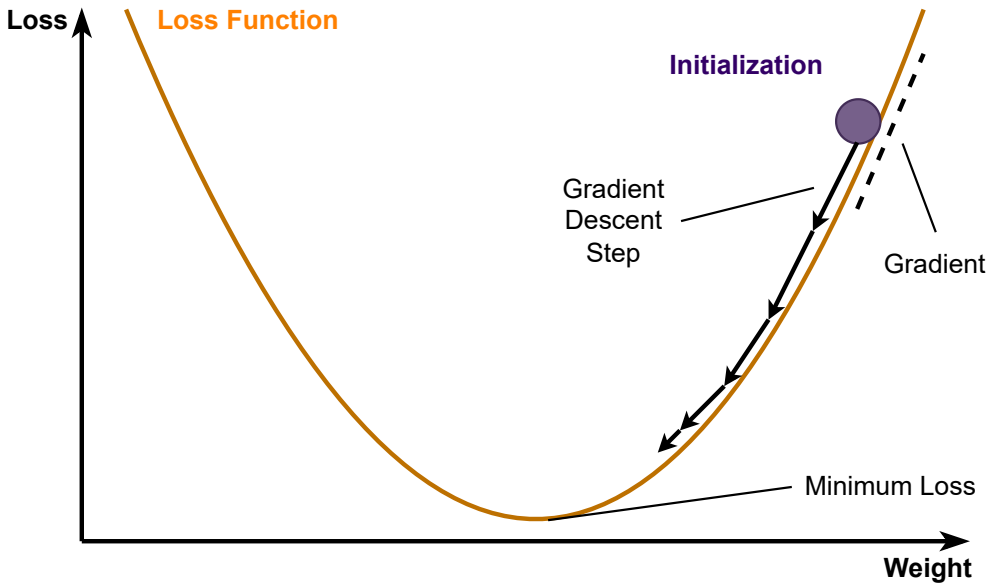


Figure 17: Visualization of the gradient descent algorithm.

This is exactly what we want to do in our situation. We want to minimize our loss $\mathcal{L}$, since this means that our regression line fits the data set in an optimal way. In this case, this is done by finding an optimal $w_0^\star$ and $w_1^\star$ for which the loss reaches its minimum, as stated below:

$$w_0^\star, w_1^\star = \underset{w_0,w_1}{\operatorname{argmin}}\{\mathcal{L}\} = \underset{w_0,w_1}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^{N} (w_0 + w_1 x_i - y_i)^2 \tag{17}$$

One step of the gradient descent optimization algorithm for a parameter θ of a

function $f(\theta)$ is given by:

$$\theta^{(t+1)} = \theta^{(t)} - lr * \frac{\partial f}{\partial \theta^{(t)}} \tag{18}$$

ToDo

1. Implement the method `gradient_descent_update_step` which applies one step of the gradient descent algorithm given the parameters:
   - `theta:` Parameter θ.
   - `theta_derivative:` Derivative $\frac{\partial f}{\partial \theta}$ of the function $f(\theta)$ w.r.t. to θ.
   - `lr:` Learning rate of the gradient descent algorithm.

Attention

Please note that this is an easy example and in reality this naive implementation of gradient descent can cause issues which may lead to non-ideal solutions. You will learn more about this in the lecture.

10.3.4 Let's Put Everything Together and Train Our Model [1 point]

Now we have all requirements to be able to train our linear regression model and iteratively fit a linear line to our data set. Let's do it!

ToDo

1. Implement the method `train` which trains our linear regression model to properly fit a linear line to the given data set, given the parameters:
   - `x:` x-values of the samples as a numpy array.
   - `y:` y-values of the samples as a numpy array.
   - `w0:` y-offset of the current regression line.
   - `w1:` Slope of the current regression line.
   - `epochs:` Number of iteration steps.
   - `lr:` Learning rate of the gradient descent algorithm.

10.4 Testing on a Real World Example [3 points]

Previously we have seen that linear regression can be used to fit a hyperplane to an artificial nD dataset. Out of simplicity, we have focused on regressing a line to a 2D dataset. However, this example can easily be up-scaled to higher dimensions. This is cool, but what can we do with that insight and why or when should we actually use linear regression? To answer these questions, we will look at the following ”real” world scenario:
Imagine it is the year 2016. You are an American citizen and live together with your partner and your two kids. You’ve just finished building your new house which can be seen in Figure 18:



Figure 18: (Photo by Pixabay: https://www.pexels.com/photo/white-and-brown-concrete-bungalow-under-clear-blue-sky-210617/)

but haven’t moved in yet. You work hard as a computer scientist, so you can easily afford such a nice house with a huge front lawn in a beautiful neighborhood. You genuinely like your life in the US which includes football on Sundays, groceries shopping at Walmart and from time to time some fast food at Chipotle. However, a couple of weeks ago you have been on vacation in Germany and have visited a friend who is living in Bamberg (Bavaria). And you have been in love with this wonderful city ever since. You admire the green spaces, the clean air, the ”Biergärten”, and especially the ”Rauchbier”. Your family felt the same way and hence, you started making plans to leave the US and move to Bamberg. Over the last years you saved some money, so you are able to afford a living in Bamberg at least for some years without the need of selling your American house directly. Since you don’t need the money immediately, you wonder whether it makes sense to keep the house for a

while and sell it later. Luckily for you have heard something about linear regression during your studies. This knowledge will come in handy now.
You study the median sales price (in US$) of new homes sold in the United States from 1963 to 2016 which you found here. An excerpt of the resulting table can be seen in Table 1.

Year	1963
1963	$18,000
1964	$18,900
1965	$20,000
⋮	⋮
1970	$23,400
⋮	⋮
1980	$64,600
⋮	⋮
1990	$122,900
⋮	⋮
2000	$169,000
⋮	⋮
2010	$221,800
⋮	⋮
2014	$282,800
2015	$296,400
2016	$316,200

Table 1: Median sales price (in US$) of new homes sold in the United States from 1963 to 2016 (data from: https://www.bls.gov/opub/btn/volume-7/housing-and-expenditures-before-during-and-after-the-bubble.htm)

You want to use these past sales prices to predict the median sales price for new homes in the year 2021. This year marks the point in time when your savings will be gone, and you definitely need to sell your former house. You will use linear regression to do that. Let's get started.

10.4.1 Load the Data [1 point]

As the first step we have to load the data about the median sales price (in US$) of new homes sold in the United States from 1963 to 2016.

ToDo

1. Implement the method `load_real_world_dataset` by using your `DataLoader` class to load the data.

10.4.2 Run the Linear Regression Algorithm [1 point]

Now we can regress a linear line for our real world dataset.

ToDo

1. Implement the method `train_for_real_world_dataset` which trains a linear regression model on the real world dataset:
   (a) Use the previously implemented method `load_real_world_dataset` to load the real world dataset.
   (b) Use the previously implemented method `train` to train a linear regression model for our real world dataset.

10.4.3 Calculate the Expected Median Sales Price in 2021 [1 point]

We have successfully regressed a linear line to the median sales price (in US$) of new homes sold in the United States from 1963 to 2016. We now want to use the parameters of that line (y-offset and slope) to calculate the expected median sales price (in US$) of new homes sold in the United States in 2021.

ToDo

1. Implement the method `predict` to predict the expected median sales price (in US$) of new homes sold in the United States in a particular future year.

10.4.4 Finishing Up [0 point]

We have predicted the expected median sales price (in US Dollar) of new homes sold in the United States in 2021 based on our linear regression algorithm which

we have executed before. We have accounted for all sales prices from 1963 up to 2016. Since we use so many samples, we end up with a potential robust estimation against financial crises or other effects. However, as you can see we are not able to regress the trend within last couple of years (starting 2010) properly. We drastically underestimate that trend which leads to the fact that our predicted median sales price for 2021 is lower than the sales price for 2016.
You could continue in two different ways from here. First, sell the house directly and don't bother yourself with it anymore. Second, adjust your regression problem to consider only the years starting 2000 (or starting 2010), since these represent the current trend in housing prices better than the samples in the 60s up to the 90s. You could regress a new linear line for these years and check again whether it makes sense to keep the house and sell later, or to sell the house directly. However, always remember that your house won't be new anymore in 2021, so you are not able to sell it for the maximum amount of money. And be aware that the housing bubble could burst and the housing prices could drop dramatically any moment. So what would you do?

11 Logistic Regression [13 points]

In the last exercise we have seen how linear regression can be used to fit a hyperplane (for the 2D case this is a line) to a set of points by minimizing the distances of these points to that hyperplane. This can be used to predict for example the trend in stock, or housing prices based on last observations. However, for other applications, like the separation of two different classes of data points (classification), linear regression can no longer be used to a satisfying extent. Here we rely on another regression model, the so-called logistic regression. Logistic regression uses the logistic (sigmoid) function to map the output of a linear equation in between the value range $[0,1]$ as it can be seen in Figure 19:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

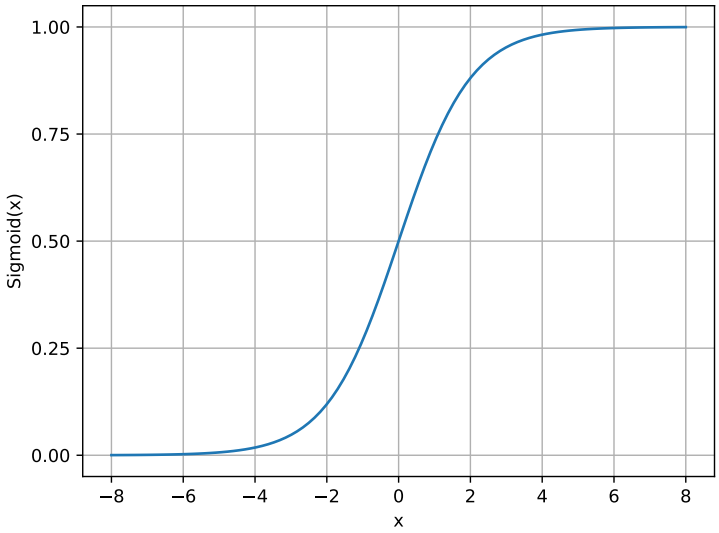


Figure 19: Equation and Visualization of the Sigmoid function.

These output values between 0 and 1 created by the logistic regression algorithm can further be understood as probabilities. This is exactly why logistic regression works well for a 2-class classification problem. The output probabilities can be used to threshold two different classes and assign new samples (data points) to either one class or the other. This fact can be seen in Figure 20.
The probabilities for all samples of "Class 0" (green circles) and "Class 1" (red triangles) were calculated using the sigmoid function. We can now use the probability of e.g. 0.5 to threshold "Class 0" from "Class 1" as well as classify new samples to either one or the other class depending on their probability (p(new sample)):

$$\text{Class(new sample)} = \begin{cases} 0 & \text{if } p(\text{new sample}) < 0.5 \\ 1 & \text{else} \end{cases} \tag{19}$$

Hence, logistic regression represents our very first classification algorithm, and we will implement it in this exercise.

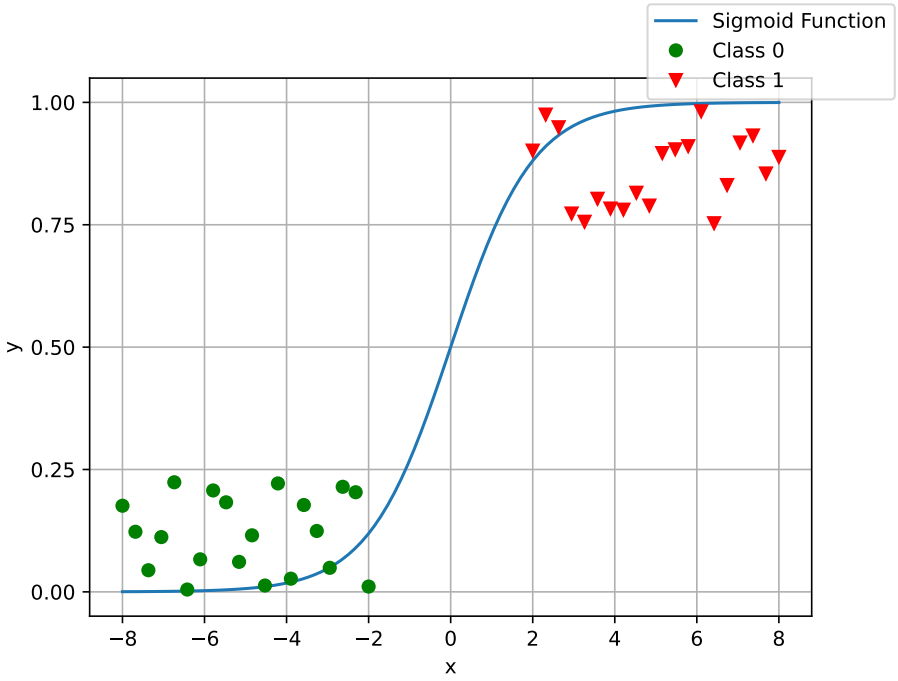


Figure 20: Thresholding two classes by using logistic regression.

ToDo

1. Follow the instructions in `logistic_regression.ipynb` to finish this exercise.

11.1 Data Loading [3 points]

The first task of this exercise is to load the dataset.

ToDo

1. Implement the class `DataLoader`.
2. Implement the constructor `__init__` which initializes our data loader.
3. Implement the method `load_images` which loads all cat and dog images, in file-name order, and converts them to numpy arrays.
4. Implement the method `initialize_classes` which initializes the classes as a dictionary.

5. Implement the method `initialize_labels` which initializes the labels for our two classes cats and dogs.
6. Implement the method `split_images_and_labels_into_train_and_test_sets` which splits all images and their corresponding labels into a training and test set.
7. Implement the method `load_train_and_test_sets` which runs all previous functions to create the training and test sets.
8. Implement the method `get_train_and_test_sets` which returns the train and test sets.

11.2 Preprocessing [2 points]

The second task of this exercise is to preprocess the dataset. Preprocessing can include:

- Resizing the inputs, so that their dimensions match
- Reshaping the inputs, so that calculations can be performed more efficiently
- Cropping the inputs, so that unnecessary information is deleted or the data dimensionality is reduced (enable computations)
- Centering/Standardizing/Normalizing the inputs
- ...

You will implement some of these methods in the code block below.

ToDo

1. Implement the class `Preprocessor`.
2. Implement the method `resize_images` which resizes all images from their original shape to a new shape. Keep the pixel values in $[0, 255]$ (`preserve_range=True`); `normalize_images` scales them to $[0, 1]$ afterwards.
3. Implement the method `reshape_sets` which reshapes the sets from lists to np.arrays.
4. Implement the method `flatten_images` which flattens the train and test images over the spatial dimension.
5. Implement the method `normalize_images` which normalizes the train and test image sets.
6. Implement the method `preprocess` which runs all previous functions to preprocess the training and test sets.

11.3 Building the Algorithm [7 points]

The third task of this exercise is to build the individual parts of our algorithm. For that you will build a Logistic Regression, using a Neural Network mindset. Figure 21 shows why the Logistic Regression algorithm is indeed a very simple Neural Network.

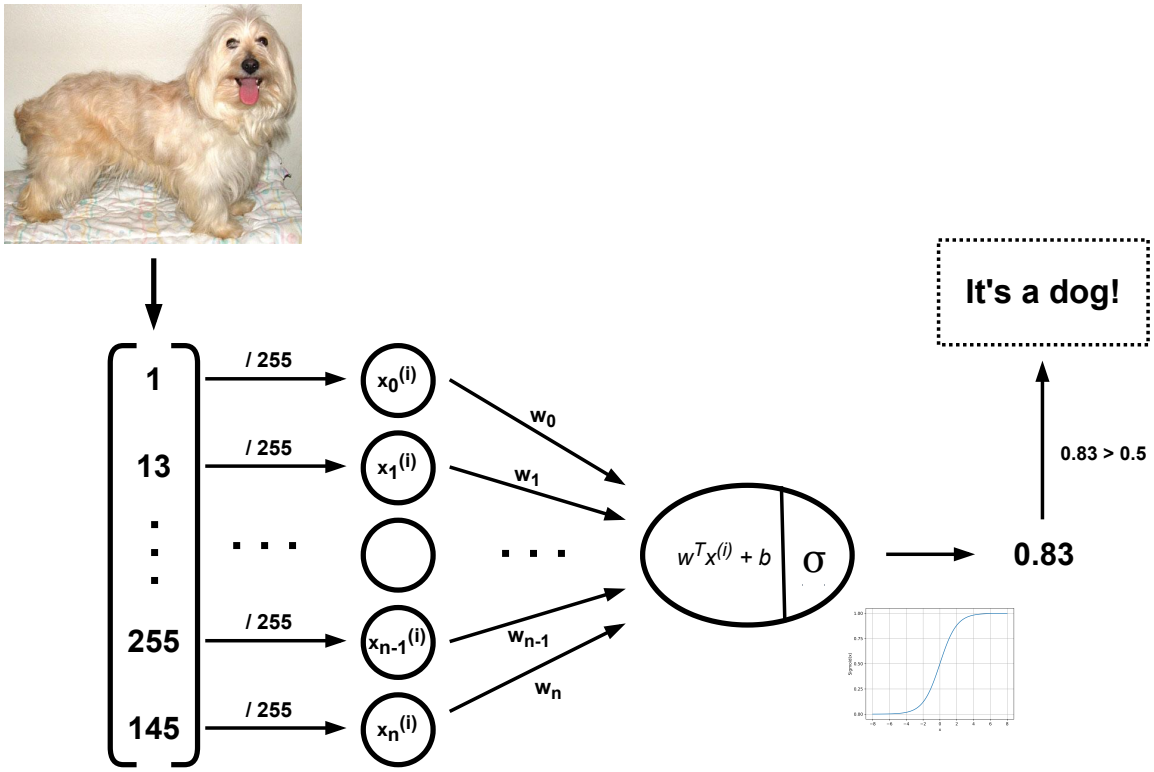


Figure 21: Logistic regression in the fashion of a simple neural network.

Underlying math of the algorithm for one sample $x^{(i)}$:

$$\begin{aligned} z^{(i)} &= w^T x^{(i)} + b \\ \hat{y}^{(i)} &= a^{(i)} = sigmoid(z^{(i)}) \\ \mathcal{L}(a^{(i)}, y^{(i)}) &= -y^{(i)} \log(a^{(i)}) - (1 - y^{(i)}) \log(1 - a^{(i)}) \end{aligned} \tag{20}$$

And the cost is then computed by summing over all training samples m like this:

$$J = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(a^{(i)}, y^{(i)}) \tag{21}$$

11.3.1 Activation Function [1 point]

For the first part of our algorithm, we will implement the activation function, which will be the sigmoid function.

Remember the sigmoid function can be compted like this:

$$\sigma(x) = \frac{1}{1+e^{-x}} \tag{22}$$

ToDo

1. Implement the method `sigmoid` which calculates the sigmoid activation function for an input `x`.

11.3.2 Parameter Initialization [0.75 point]

In the next step we will initialize our “network” parameters. For simplicity reasons we will use zero initialization here. However, be advised that this is normally no ideal solution.

ToDo

1. Implement the method `initialize_with_zeros` which initializes the weights of the network ”w” as a vector of zeros and the bias ”b” as 0.

11.3.3 Forward and Backward Propagation [1 point]

Now that your parameters are initialized, we can do the ”forward” and ”backward” propagation steps for learning the parameters.
Forward propagation (Compute the cost for the current input based on the current parameters):

- You get the input X
- You compute $A = \sigma(w^T X + b) = (a^{(1)}, a^{(2)}, ..., a^{(m-1)}, a^{(m)})$
- You calculate the cost function: $J = -\frac{1}{m}\sum_{i=1}^{m}(y^{(i)}\log(a^{(i)}) + (1-y^{(i)})\log(1-a^{(i)}))$

Backward propagation (Calculate the gradients of the parameters based on the current input):

- $\frac{\partial J}{\partial w} = \frac{1}{m}X(A-Y)^T$
- $\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(a^{(i)} - y^{(i)})$

ToDo

1. Implement the method `forward` which calculates the forward propagation of our model.
2. Implement the method `backward` which calculates the backward propagation of our model.

11.3.4 Optimization [0.5 point]

So far, we initialized our parameters, computed the cost as well as the gradients of our cost with respect to our parameters. Now we need to optimize the latter to ensure that our model performs better and better over time. For that we will update the parameters using gradient descent.
The goal is to learn w and b by minimizing the cost function J. We know that the update rule of gradient descent for a parameter $\theta^{(t)}$ is defined as:

$$\theta^{(t+1)} = \theta^{(t)} - lr * \frac{\partial J}{\partial \theta^{(t)}} \tag{23}$$

where lr is the learning rate.

ToDo

1. Implement the method `optimize` which optimizes 'w' and 'b' by running the gradient descent algorithm.

11.3.5 Prediction [0.75 point]

The last part will ensure that our model keeps improving over time. But so far we are not able to see any results of that. Our model is not doing anything yet. Hence, in this part we will implement the prediction of the model. In our case, we want that our model predicts either a cat or a dog for a given image.

ToDo

1. Implement the method `predict` which predicts whether the label is 0 or 1 using the learned logistic regression parameters (w, b).

11.3.6 Merging all Functions into a Model [3 points]

We have implemented all individual parts of our algorithm. The last step that is missing is to merge all of these functions into an executable model.

ToDo

1. Implement the method `model` which creates the logistic regression model.

11.4 Viewing and Evaluating the Results [1 point]

If you successfully pass all the tests before, let's have a look at our results.

ToDo

1. Implement the method `evaluate` which puts everything together: it loads the dataset, preprocesses it, trains the model and creates predictions for the unseen test images.

Comment:
Look at the two accuracies. The model classifies every training image correctly (100%), but only 58% of the unseen test images, and with 50 test images that is not far from guessing. The model has memorised the training set instead of learning what distinguishes a cat from a dog: it **overfits**.

This behavior has multiple reasons. First, the model has 12,289 parameters (one weight per pixel value, plus the bias) but only 200 training images. With that many parameters it can separate the training images perfectly using pixel patterns that do not carry over to new images.
Second, logistic regression is a linear model on raw pixels. Whether an image shows a cat or a dog is not a linear function of its pixel values: position, pose, lighting and background change the pixels far more than the class does. To learn features that are robust to such variation, the model needs more capacity. Maybe we need a more powerful (deeper) model ;)
Third, nothing in our training discourages memorisation. Techniques such as regularization, more data or data augmentation all counteract it.

In the next assignments, we will look into these things and come up with better ways of classifying data like images than the simple logistic regression we have seen here.